☁️ บทที่ 1: Introduction to Cloud Computing

(พื้นฐานการประมวลผลบนคลาวด์)

1. Cloud Computing คืออะไร?

Cloud Computing คือการส่งมอบทรัพยากรด้านไอที (IT Resources) ตามความต้องการผ่านอินเทอร์เน็ต โดยคิดค่าใช้จ่ายตามการใช้งานจริง (Pay-as-you-go) [1]แนวคิดนี้เปลี่ยนมุมมองจากการมองโครงสร้างพื้นฐานเป็น "ฮาร์ดแวร์" (Hardware) ให้กลายเป็น "ซอฟต์แวร์" (Infrastructure as Software) ซึ่งมีความยืดหยุ่นสูงกว่ามาก [2]

2. รูปแบบการให้บริการ (Service Models)

เอกสารนำเสนอโมเดลแบบพีระมิด โดยไล่ระดับจากการควบคุมมากไปน้อย ดังนี้:

- **IaaS (Infrastructure as a Service):** ผู้ใช้เช่าโครงสร้างพื้นฐาน เช่น เซิร์ฟเวอร์, สตอเรจ, เครือข่าย ผู้ใช้มีอำนาจควบคุมสูงสุด (เสมือนมี Data Center เอง) *ตัวอย่างเสริม: AWS EC2, Google Compute Engine*
- **PaaS (Platform as a Service):** ผู้ให้บริการเตรียม Hardware และ Software Environment ให้ (เช่น OS, Database) ผู้ใช้โฟกัสแค่การเขียนโค้ดและแอปพลิเคชัน [4][4][4][4] *ตัวอย่างเสริม: Heroku, Google App Engine*
- **SaaS (Software as a Service):** ซอฟต์แวร์สำเร็จรูปที่ใช้งานผ่านอินเทอร์เน็ต ผู้ใช้ไม่ต้องดูแลอะไรเลยนอกจากใช้งาน *ตัวอย่างเสริม: Gmail, Google Drive, Salesforce*

3. รูปแบบการติดตั้ง (Deployment Models)

- **Public Cloud:** ทรัพยากรเป็นของผู้ให้บริการ เปิดให้คนทั่วไปเช่าใช้ [6]
- **Private Cloud:** ระบบคลาวด์ที่สร้างขึ้นใช้ภายในองค์กรเดียว เพื่อความปลอดภัยสูงสุด [7]
- **Hybrid Cloud:** การใช้งานร่วมกันระหว่าง Public และ Private Cloud [8]

4. ข้อดี 6 ประการของ Cloud Computing [9]

1. **เปลี่ยนรายจ่าย**: เปลี่ยนเงินลงทุนก้อนใหญ่ (Capital Expense) เป็นค่าใช้จ่ายผันแปรตามจริง (Variable Expense)
2. **ประหยัดจากขนาด (Economies of Scale):** ยิ่งมีผู้ใช้มาก ต้นทุนต่อหน่วยยิ่งถูกลง
3. **ไม่ต้องเดาความจุ**: ไม่ต้องซื้อเผื่อ (Over/Under-provisioning) สามารถขยายหรือลด (Scale) ตามการใช้งานจริง
4. **เพิ่มความเร็วและความคล่องตัว**: ลดเวลาเตรียมเซิร์ฟเวอร์จากเป็นสัปดาห์เหลือเพียงไม่กี่นาที
5. **ลดภาระดูแล Data Center:** ไม่ต้องเสียเงินค่าไฟ ค่าที่ หรือทีมงานดูแลฮาร์ดแวร์
6. **ขยายไปทั่วโลกในไม่กี่นาที**: สามารถ Deploy แอปพลิเคชันไปหลาย Region ทั่วโลกได้อย่างรวดเร็ว

🐳 บทที่ 2: Introduction to Docker

(ทำความรู้จักกับ Docker)

1. Docker คืออะไร?

Docker เป็นแพลตฟอร์มซอฟต์แวร์แบบ **OS-level virtualization** ที่ช่วยให้นักพัฒนาสร้าง (Build), นำไปใช้ (Deploy), และรัน (Run) แอปพลิเคชันในรูปแบบของ **Container** ซึ่งบรรจุทุกสิ่งที่จำเป็น (Code, Libraries, Dependencies) ไว้ในนั้น [10]

2. Docker vs. Virtual Machine (VM)

นี่คือจุดที่สำคัญที่สุดในการทำความเข้าใจ Docker:

- **Architecture:** VM ต้องมี Guest OS ในทุกๆ เครื่องจำลอง ทำให้กินทรัพยากรมาก แต่ Docker ใช้ **Docker Engine** ทำงานอยู่บน Host OS เดียวกัน และแชร์ Kernel ร่วมกัน
- **ประสิทธิภาพ**: Docker ใช้แรมน้อยกว่า (Low Memory usage), บูตเครื่องเร็วกว่า (วินาที vs นาที), และมีประสิทธิภาพสูงกว่า VM [12]

3. องค์ประกอบหลักของ Docker (Docker Architecture)

- **Docker Client (CLI):** เครื่องมือที่ผู้ใช้พิมพ์คำสั่ง (เช่น docker build, docker run) เพื่อสั่งงาน
- **Docker Daemon (Server):** ตัวทำงานเบื้องหลัง คอยจัดการ Image, Container, Network
- **Docker Registry:** สถานที่เก็บ Image (เช่น Docker Hub) [14]
- **Docker Images:** แม่พิมพ์ (Template) แบบ Read-only สำหรับสร้าง Container [15]
- **Docker Container:** อินสแตนซ์ที่รันอยู่จริง (Runnable instance) ของ Image [16]

📄 **บทที่ 3: Dockerfile** (การสร้าง Image ด้วยตนเอง)

1. Dockerfile คืออะไร?

คือไฟล์ Text document ที่บรรจุชุดคำสั่ง (Instruction) ทีละขั้นตอน เพื่อใช้ประกอบร่าง (Assemble) สร้างเป็น Docker Image [17]

2. คำสั่งสำคัญใน Dockerfile

คำสั่ง	หน้าที่	ข้อมูลเพิ่มเติม (เสริม)
FROM	ระบุ Base Image เริ่มต้น (เช่น FROM node:latest)	ต้องเป็นคำสั่งแรกเสมอ
WORKDIR	กำหนดโฟลเดอร์ทำงาน (Working Directory)	เหมือนคำสั่ง cd ใน Linux
COPY	คัดลอกไฟล์จากเครื่องเราเข้าไปใน Image	ใช้บ่อยในการย้าย Code เข้าไป
RUN	สั่งรันคำสั่งขณะ Build Image	เช่น npm install หรือ apt-get install
EXPOSE	แจ้งว่าจะเปิด Port อะไร	เป็นเพียง Metadata เพื่อบอกผู้ใช้
CMD	คำสั่ง Default ที่จะรันเมื่อ Container เริ่มทำงาน	สามารถถูกทับได้ตอนสั่ง docker run

3. วงจรการทำงาน (Workflow)

1. เขียน **Dockerfile**
2. ใช้คำสั่ง docker build -t <ชื่อภาพ> เพื่อสร้าง **Image**
3. ใช้คำสั่ง docker run เพื่อสร้าง **Container** จาก Image นั้น
4. ใช้คำสั่ง docker push เพื่อนำ Image ไปฝากไว้บน **Registry**

🐙 **บทที่ 4: Docker Compose**

(การจัดการ Multi-Container)

1. Docker Compose คืออะไร?

เครื่องมือสำหรับนิยามและรันแอปพลิเคชันที่ประกอบด้วย **หลาย Container** (Multi-container applications) โดยใช้ไฟล์เพียงไฟล์เดียว (YAML file) [20] เหมาะสำหรับระบบที่มีความซับซ้อน เช่น Web Server เชื่อมต่อกับ Database

2. โครงสร้างไฟล์ compose.yaml [21]

ไฟล์จะเขียนในรูปแบบ YAML โดยมีองค์ประกอบหลักคือ:

- **version:** ระบุเวอร์ชันของ Compose file schema
- **services:** ระบุ Container ต่างๆ ที่จะใช้งาน (เช่น web, db)
    - image: ระบุ Image ที่ใช้
    - ports: การ Map port (Host:Container)
    - volumes: การแชร์ไฟล์ระหว่างเครื่อง Host กับ Container
    - depends_on: ระบุลำดับการเริ่มทำงาน (เช่น ให้ Database รันก่อน Web) [22]

3. คำสั่งสำคัญ [23]

- docker-compose up -d: สร้างและเริ่มรันทุก Service ในโหมด Background
- docker-compose ps: ดูสถานะของ Container ในระบบ Compose
- docker-compose stop: หยุดการทำงาน
- docker-compose down: หยุดและลบ Container/Network ที่สร้างขึ้น (ข้อมูลเสริม)

4. ตัวอย่างการใช้งานจริง (Demo)

ในเอกสารยกตัวอย่างการทำ **PHP เชื่อมต่อ MySQL**:

- ใช้ php:8.2-apache เป็น Web Server
- ใช้ mysql:8.0 เป็น Database
- เชื่อมต่อกันผ่าน Network ภายในที่ Docker Compose สร้างให้โดยอัตโนมัติ โดยโค้ด PHP สามารถเรียก Hostname ว่า "db" ได้เลยตามชื่อ Service [24]

💡 **ข้อมูลเสริม (External Knowledge Insight)**

เพื่อความเข้าใจที่ลึกซึ้งยิ่งขึ้น ผมขอสรุปเกร็ดความรู้เพิ่มเติมที่ไม่ได้ระบุชัดเจนในสไลด์แต่สำคัญมาก:

1. **Layer Caching ใน Dockerfile:**
    - ทุกคำสั่ง (RUN, COPY) ใน Dockerfile จะสร้าง "Layer" ใหม่
    - Docker ฉลาดพอที่จะจำ Cache ของ Layer เก่าไว้ ถ้าคุณแก้บรรทัดล่างๆ Docker จะดึง Cache บรรทัดบนมาใช้ ทำให้การ Build ครั้งต่อไปเร็วมาก ดังนั้น **ควรเอาคำสั่งที่เปลี่ยนแปลงบ่อย (เช่น Copy Code) ไว้ล่างสุด** และเอาคำสั่งที่ไม่ค่อยเปลี่ยน (เช่น Install Library) ไว้บนๆ
2. **CMD vs ENTRYPOINT:**
    - ในสไลด์พูดถึง CMD ว่าเป็นคำสั่ง Default [25]
    - *ข้อมูลเสริม:* ENTRYPOINT จะถูกใช้เมื่อต้องการให้ Container ทำงานเป็น Executable (เปลี่ยนคำสั่งไม่ได้ง่ายๆ) ส่วน CMD มักใช้เป็นพารามิเตอร์เสริมหรือคำสั่งที่ผู้ใช้เปลี่ยนทับได้ง่ายๆ ตอนสั่ง Run
3. **ทำไมต้องใช้ Docker Compose?**
    - ถ้าใช้แค่ Docker CLI ปกติ คุณต้องพิมพ์คำสั่งยาวๆ (docker run -d -p 80:80 --network mynet ...) หลายรอบสำหรับทุก Container
    - Docker Compose ช่วยให้คุณพิมพ์แค่ docker-compose up คำสั่งเดียว ระบบทุกอย่าง (Web, DB, Cache) ก็จะขึ้นมาพร้อมกันตามคอนฟิกที่เซ็ตไว้